COMPUTER ENGINEERING

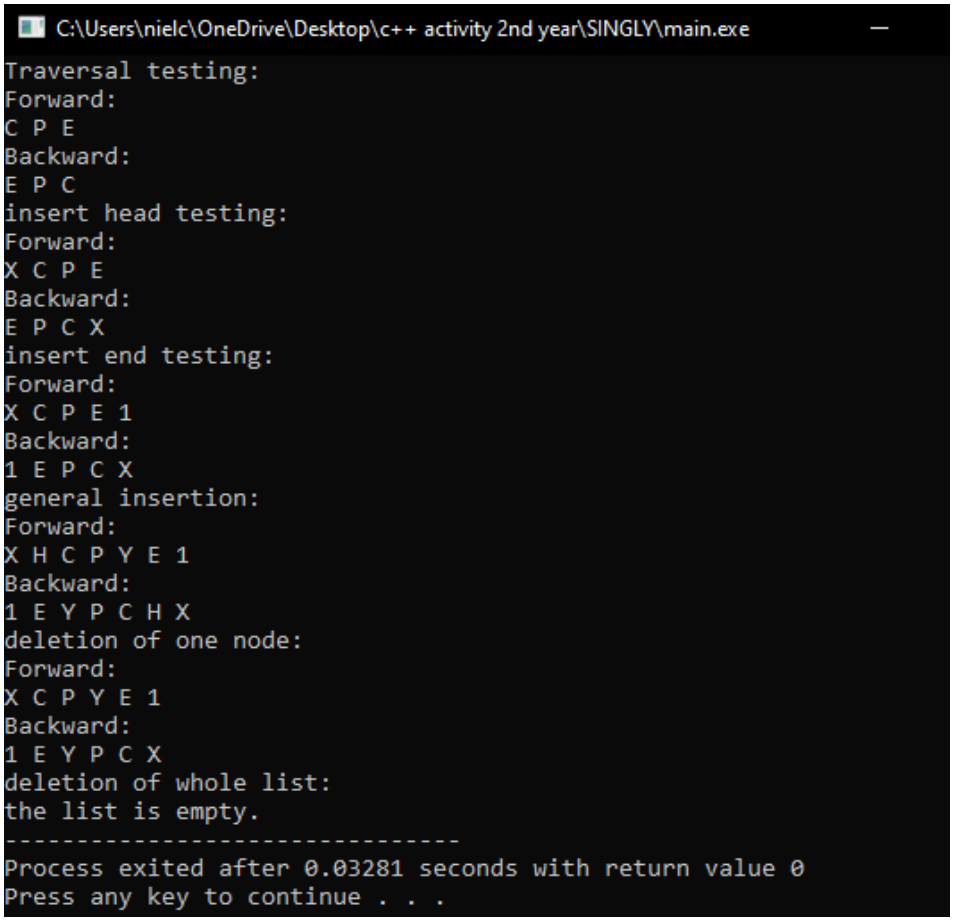
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	07/23/2026	**DATE SUBMITTED:**	07/28/2026
NAME:	Cabug, Jeffordniel H.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS-ON ACTIVITY NO. 3.1 LINK LIST

1. PROCEDURE OUTPUT

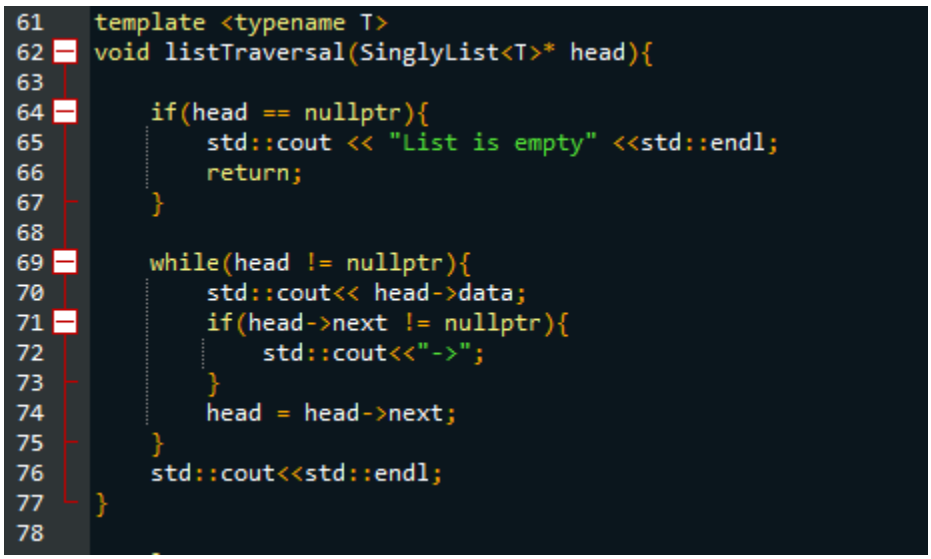
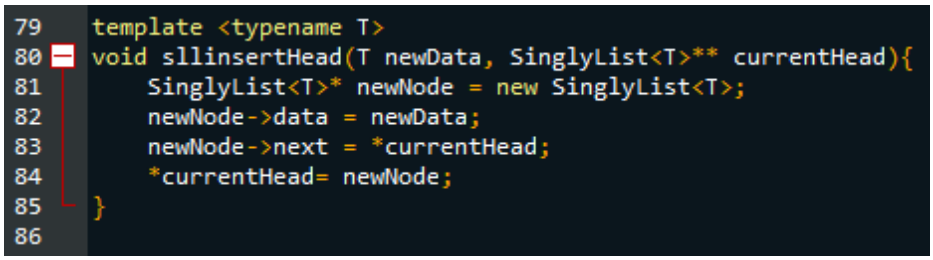
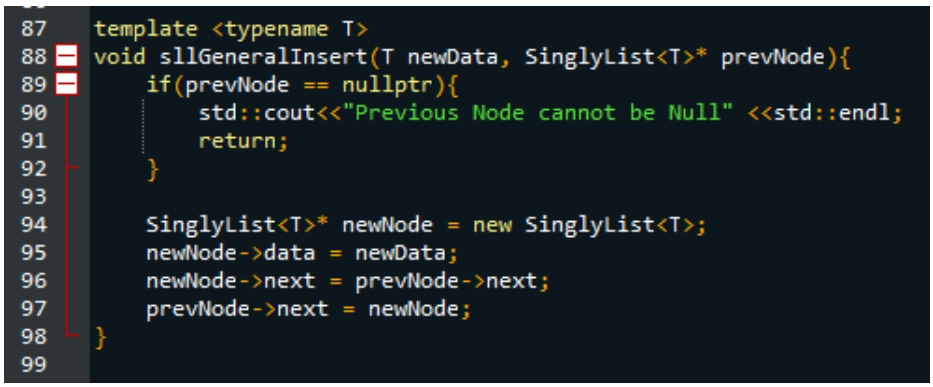
Screenshot



Discussion

The output shows that the doubly linked list program is working properly. It first displays the list forward and backward, showing that the nodes are connected correctly. After adding **X** at the beginning, **1** at the end, and **H** in the middle, the list updates correctly each time. When **H** is deleted, the list returns to its correct order. Finally, deleting the whole list shows the message **"the list is empty,"** which means all the nodes were removed successfully.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	
Insertion at head	
Insertion at any part of the list	

```
61  template <typename T>
62  void listTraversal(SinglyList<T>* head){
63
64      if(head == nullptr){
65          std::cout << "List is empty" <<std::endl;
66          return;
67      }
68
69      while(head != nullptr){
70          std::cout<< head->data;
71          if(head->next != nullptr){
72              std::cout<<"->";
73          }
74          head = head->next;
75      }
76      std::cout<<std::endl;
77  }
78
```

```
79  template <typename T>
80  void sllinsertHead(T newData, SinglyList<T>** currentHead){
81      SinglyList<T>* newNode = new SinglyList<T>;
82      newNode->data = newData;
83      newNode->next = *currentHead;
84      *currentHead= newNode;
85  }
86
```

```
87  template <typename T>
88  void sllGeneralInsert(T newData, SinglyList<T>* prevNode){
89      if(prevNode == nullptr){
90          std::cout<<"Previous Node cannot be Null" <<std::endl;
91          return;
92      }
93
94      SinglyList<T>* newNode = new SinglyList<T>;
95      newNode->data = newData;
96      newNode->next = prevNode->next;
97      prevNode->next = newNode;
98  }
99
```

Insertion at the end

```
100 template <typename T>
101 void sllinsertEnd(T newData, SinglyList<T>** head){
102     SinglyList<T>* newNode = new SinglyList<T>;
103     SinglyList<T>* currentNode = *head;
104     newNode->data = newData;
105     newNode->next = nullptr;
106 
107     while(currentNode->next != nullptr){
108         currentNode = currentNode->next;
109     }
110 
111     currentNode->next = newNode;
112 
113 }
114 
```

Deletion of a node

```
115 template <typename T>
116 void sllDeleteNode(T findData, SinglyList<T>** head){
117     if(*head == nullptr) return;
118 
119     SinglyList<T>* currNode = *head;
120     SinglyList<T>* prevNode = nullptr;
121 
122     while(currNode != nullptr && currNode->data != findData){
123         prevNode = currNode;
124         currNode = currNode->next;
125     }
126 
127     if(currNode == nullptr) return;
128 
129     if(prevNode == nullptr){
130         *head = currNode->next;
131     }else{
132         prevNode-> next = currNode->next;
133     }
134 
135     delete currNode;
136 
137 
138 }
```

Table 3-2. Code for the List Operations

a.	**Source Code**	`std::cout<<"Traversal testing: " <<std::endl;` `dllTraverse(head);`

	Console	`Traversal testing:`
		`Forward:`
		`C P E`

b. **Source Code**

```
std::cout<<"insert head testing: "<< std::endl;
dllInsertHead('X', &head);
dllTraverse(head);
```

Console

```
Traversal testing:
Forward:
C P E
Backward:
E P C
```

c. **Source Code**

```
std::cout<<"insert end testing: "<< std::endl;
dllInsertEnd('1', head);
dllTraverse(head);
```

Console

```
insert head testing:
Forward:
X C P E
Backward:
E P C X
```

d. **Source Code**

```
std::cout<<"general insertion: "<<std::endl;
dllGenIns('H', head);
dllGenIns('Y', second);
dllTraverse(head);
```

Console

```
general insertion:
Forward:
X H C P Y E 1
Backward:
1 E Y P C H X
```

e. **Source Code**

```
std::cout<<"deletion of one node: "<<std::endl;
dllDeleteNode('H', &head);
dllTraverse(head);
```

Console

```
deletion of one node:
Forward:
X C P Y E 1
Backward:
1 E Y P C X
```

f.	**Source Code**	`std::cout<<"deletion of whole list: "<<std::endl;`
		`dllDeleteList(&head);`
		`dllTraverse(head);`

	Console	`deletion of whole list:` `the list is empty.`

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots(s)	Analysis

`template<typename T>`
`void dllTraverse(DoubleList<T> *currentNode){`
`  if(currentNode == nullptr){`
`  std::cout <<"the list is empty.";`
`  return;`
`  }`
 	
`  DoubleList<T>* tail;`
 	
`  std::cout<<"Forward: " <<std::endl;`
`  while(currentNode != nullptr){`
`  std::cout<<currentNode->data<<" ";`
`  tail = currentNode;`
`  currentNode = currentNode->next;`
`  }`
 	
`  std::cout<<"\nBackward: " <<std::endl;`
`  while(tail != nullptr){`
`  std::cout<<tail->data<<" ";`
`  tail = tail->prev;`
`  }`
`  std::cout<<std::endl;`
 	
`}`
	The following code presents a function that traverses the implemented doubly linked list in both the forward and backward directions, beginning from the node passed as the parameter currentNode.

`template<typename T>`
`void dllInsertHead(T newData, DoubleList<T>** currentHead){`
`  DoubleList<T>* newNode = createNode(newData);`
`  newNode->next = *currentHead;`
`  if(*currentHead != nullptr){`
`  (*currentHead)->prev = newNode;`
`  }`
`  *currentHead = newNode;`
 	
`}`
	This function, dllInsertHead, inserts a new node at the beginning of a doubly linked list while also checking whether the list is empty before performing the insertion.

```cpp
template<typename T>
void dllInsertEnd(T newData, DoubleList<T> *currentHead){
    DoubleList<T> *newNode = createNode(newData);
    while(currentHead->next !=nullptr){
        currentHead = currentHead->next;
    }

    currentHead->next = newNode;
    newNode->prev = currentHead;
}

```

This function inserts a new node at the tail of the doubly linked list, unlike the previous function, which adds a node to the head. The process is largely similar: it creates a new node, traverses the list until it reaches the last node (where the next pointer is null), and then links the new node to the end of the list by updating the appropriate pointers.

```cpp
template <typename T>
void dllGenIns(T newData, DoubleList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be empty" << std::endl;
        return; // Prevents dereferencing nullptr below
    }

    // Create new node
    DoubleList<T>* newNode = createNode(newData);

    // Link pointers
    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    if (prevNode->next != nullptr) {
        prevNode->next->prev = newNode; // Connect the existing next node back to newNode
    }

    prevNode->next = newNode;
}

```

This function inserts a new node immediately after a specified node in the doubly linked list. It also includes a validation check to prevent insertion at the front of the list, where the previous pointer would be null, helping avoid potential errors during execution.

```cpp
template <typename T>
void dllDeleteNode(T findData, DoubleList<T>** head){
    if(*head == nullptr) return;

    DoubleList<T>* currNode = *head;


    while(currNode != nullptr && currNode->data != findData){
        currNode = currNode->next;
    }


    if(currNode == nullptr) return;

    if(currNode == *head){
        *head = currNode->next;
        if(*head != nullptr){
            (*head)->prev = nullptr;
        }
    }

    else {
        currNode->prev->next = currNode->next;
        if(currNode->next != nullptr){
            currNode->next->prev = currNode->prev;
        }
    }

    delete currNode;
}
```

This function searches for a specified node in the doubly linked list and removes it by updating the surrounding node pointers and deallocating its memory. It is designed to correctly handle deletion cases involving the head, middle, and tail nodes of the list.

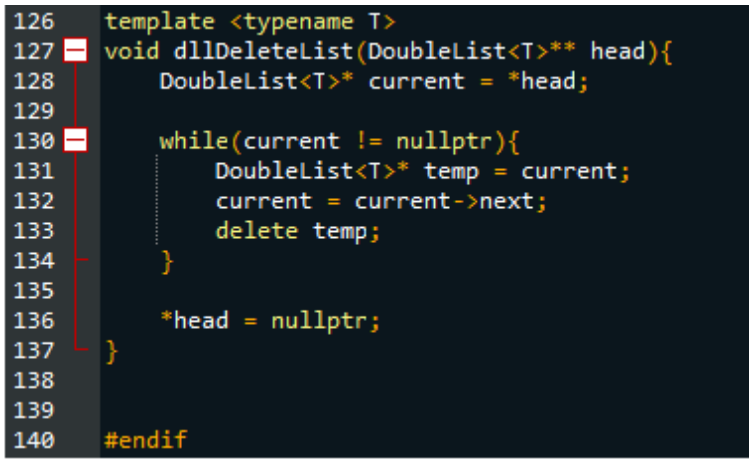
	This function, dllDeleteList, deletes every node in the doubly linked list, effectively freeing all allocated memory. It traverses the list from the head, temporarily storing the current node before moving to the next one, allowing each node to be safely deleted. Once all nodes have been removed, the head pointer is set to nullptr to indicate that the list is now empty and to prevent any dangling pointers.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.

- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

Main

```cpp
#include <iostream>
#include <string>
#include "double_LL.h"

int main() {

    doubleList<std::string>* playlist = nullptr;

    // Create Playlist
    insertSong(&playlist, std::string("Your Call"));
    insertSong(&playlist, std::string("Vulnerable"));
    insertSong(&playlist, std::string("King for a day"));
    insertSong(&playlist, std::string("Twist"));

    std::cout << "PLAYLIST\n";
    playPlaylist(playlist);

    // Current Song
    doubleList<std::string>* current = playlist;

    std::cout << "\nCurrent Song: " << current->data << std::endl;

    nextSong(&current);
    std::cout << "Next Song: " << current->data << std::endl;

    nextSong(&current);
    std::cout << "Next Song: " << current->data << std::endl;

    previousSong(&current);
    std::cout << "Previous Song: " << current->data << std::endl;

    // Add Song
    insertSong(&playlist, std::string("Shape of You"));

    std::cout << "\nPlaylist After Adding a Song\n";
    playPlaylist(playlist);

    // Remove Song
    deleteSong(&playlist, std::string("Perfect"));

    std::cout << "\nPlaylist After Removing 'Perfect'\n";
    playPlaylist(playlist);

    return 0;
}

```

DOUBLE_LL

double_LL.h | ILOB.cpp

```cpp
#ifndef DOUBLE_LL_H
#define DOUBLE_LL_H
#include <iostream>

template <typename T>
class doubleList {
public:
    T data;
    doubleList<T>* next;
    doubleList<T>* prev;
};

// Create Node
template <typename T>
doubleList<T>* newNode(const T& newData) {
    doubleList<T>* temp = new doubleList<T>;

    temp->data = newData;

    // Circular pointers
    temp->next = temp;
    temp->prev = temp;

    return temp;
}

// Insert at End
template <typename T>
void insertSong(doubleList<T>** head, const T& newData) {

    doubleList<T>* temp = newNode(newData);

    // Empty playlist
    if (*head == nullptr) {
        *head = temp;
        return;
    }

    doubleList<T>* last = (*head)->prev;

    temp->next = *head;
    temp->prev = last;

    last->next = temp;
    (*head)->prev = temp;
}

// Delete Song
template <typename T>
void deleteSong(doubleList<T>** head, T value) {

    if (*head == nullptr)
        return;
```

```
53          return;
54
55      doubleList<T>* current = *head;
56
57      do {
58
59          if (current->data == value) {
60
61              // Only one node
62              if (current->next == current) {
63                  delete current;
64                  *head = nullptr;
65                  return;
66              }
67
68              current->prev->next = current->next;
69              current->next->prev = current->prev;
70
71              if (current == *head)
72                  *head = current->next;
73
74              delete current;
75              return;
76          }
77
78          current = current->next;
79
80      } while (current != *head);
81  }
82
83  // Play Playlist
84  template <typename T>
85  void playPlaylist(doubleList<T>* head) {
86
87      if (head == nullptr) {
88          std::cout << "Playlist is empty.\n";
89          return;
90      }
91
92      doubleList<T>* current = head;
93
94      std::cout << "\nSongs in Playlist\n";
95
96      do {
97
98          std::cout << current->data << std::endl;
99          current = current->next;
100
101     } while (current != head);
102 }
103
104 // Next Song
105 template <typename T>
106 void nextSong(doubleList<T>** current) {
107
108     if (*current != nullptr)
```

```
109         *current = (*current)->next;
110     }
111
112     // Previous Song
113     template <typename T>
114     void previousSong(doubleList<T>** current) {
115
116         if (*current != nullptr)
117             *current = (*current)->prev;
118     }
119
120     #endif
```

OUTPUT:

```
Vulnerable
King for a day
Twist

Current Song: Your Call
Next Song: Vulnerable
Next Song: King for a day
Previous Song: Vulnerable

Playlist After Adding a Song

Songs in Playlist
Your Call
Vulnerable
King for a day
Twist
Shape of You

Playlist After Removing 'Perfect'

Songs in Playlist
Your Call
Vulnerable
King for a day
Twist
Shape of You
```

ANALYSIS:

Each song is stored in a node that contains the song title along with pointers to the next and previous songs, allowing users to move forward and backward through the playlist. The program includes functions for adding songs, deleting songs, displaying the playlist, and navigating between songs. It also handles special cases, such as creating the first node in an empty playlist and updating the links correctly when a song is removed.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **Chat Gpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I use ai for the concepts inside doubly functions.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.